Блок 1

Создается папка для проекта. Командой “docker network ls” просматриваются существующие сети. Командой “docker network inspect bridge” проверяются детали сети. Командой “docker network create --driver bridge app-network” создается изолированная сеть. Запускается база данных командой “docker run -d --name db --network app-network -e POSTGRES_PASSWORD=secret postgres:16-alpine”. Контейнер помещается в сеть. Запускается тестовый контейнер командой “docker run -it --rm --network app-network alpine sh”. Внутри контейнера проверяется связь командой “ping db”, имя разрешается в IP-адрес. Проверяется доступность порта командой “nc -zv db 5432”. Запускается контейнер без указания сети, команда “ping db” не работает, имя не найдется. Вывод: сети изолируют контейнеры и обеспечивают DNS.

```
sonya@fedora-4:~/docker-lab$ docker run -it --rm --network app-network alpine sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
589002ba0eae: Already exists
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
/ # ping -c 3 db
PING db (172.0.0.2): 56 data bytes
64 bytes from 172.0.0.2: seq=0 ttl=64 time=0.089 ms
64 bytes from 172.0.0.2: seq=1 ttl=64 time=0.079 ms
64 bytes from 172.0.0.2: seq=2 ttl=64 time=0.065 ms

--- db ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.065/0.077/0.089 ms
/ # nc -zv db 5432
db (172.0.0.2:5432) open
/ # exit
sonya@fedora-4:~/docker-lab$ docker run -it --rm alpine ping -c 2 db
ping: bad address 'db'
```

Блок 2

Командой “docker volume create pgdata” создается том для хранения данных. Запускается PostgreSQL командой “docker run -d --name postgres-persistent -e POSTGRES_DB=mydb -e POSTGRES_USER=user -e POSTGRES_PASSWORD=pass -v pgdata:/var/lib/postgresql/data postgres:16-alpine”. Том подключается к папке с данными базы. Создаются тестовые данные командой “docker exec -it postgres-persistent psql -U user -d mydb -c "CREATE TABLE items (id SERIAL, name TEXT); INSERT INTO items VALUES (1, 'test');"". Удаляется контейнер командой “docker rm -f postgres-persistent”. Том сохраняется. Запускается новый контейнер с тем же томом. Проверяется сохранность данных командой “docker exec postgres-restored psql -U user -d mydb -c "SELECT * FROM items;"", данные остаются на месте. Командой “docker volume inspect pgdata” просматривается путь к тому на хосте.

Блок 3
Создается структура проекта для docker-compose. Пишется файл “app.py” для backend с подключением к базе данных. Пишется файл “requirements.txt” с зависимостями. Пишется “Dockerfile” для сборки backend образа. Пишется файл “nginx.conf” для frontend чтобы перенаправлять запросы на backend. Пишется файл “docker-compose.yml” с описанием трех сервисов: db (PostgreSQL с томом и healthcheck), backend (сборка из папки, зависит от db), frontend (nginx, порт 8080, зависит от backend). Запускается весь стек командой “docker compose up -d --build”. Контейнеры создаются и запускаются.

Проверяется статус командой “docker compose ps” все сервисы становятся healthy.

```
sonya@fedora-4:~/compose-lab$ nano backend/Dockerfile
sonya@fedora-4:~/compose-lab$ docker compose up -d --build
[+] Building 15.2s (12/12) FINISHED                                  docker:default
 => [backend internal] load build definition from Dockerfile                    0.0s
 => => transferring dockerfile: 277B                                            0.0s
 => [backend internal] load metadata for docker.io/library/python:3.12-alpine  1.6s
 => [backend auth] library/python:pull token for registry-1.docker.io           0.0s
 => [backend internal] load .dockerignore                                       0.0s
 => => transferring context: 2B                                                 0.0s
 => CACHED [backend 1/6] FROM docker.io/library/python:3.12-alpine@sha256:774  0.0s
 => [backend internal] load build context                                       0.0s
 => => transferring context: 181B                                               0.0s
 => [backend 2/6] RUN apk add --no-cache wget                                   4.2s
 => [backend 3/6] WORKDIR /app                                                  0.2s
 => [backend 4/6] COPY requirements.txt .                                       0.1s
 => [backend 5/6] RUN pip install --no-cache-dir -r requirements.txt            8.1s
 => [backend 6/6] COPY app.py .                                                 0.2s
 => [backend] exporting to image                                                0.4s
 => => exporting layers                                                         0.3s
 => => writing image sha256:6cbea4f92447fd078d1b787604be7953895938f3b3e66c250  0.0s
 => => naming to docker.io/library/compose-lab-backend                          0.0s
[+] Running 3/3
 ✓ Container compose-lab-db-1        Healthy                                    0.0s
 ✓ Container compose-lab-backend-1   Heal...                                   21.8s
 ✓ Container compose-lab-frontend-1  Sta...                                    11.5s
sonya@fedora-4:~/compose-lab$ docker compose ps
NAME                     IMAGE                 COMMAND                  SERVICE    C
REATED          STATUS                    PORTS
compose-lab-backend-1    compose-lab-backend   "python app.py"          backend    3
0 seconds ago   Up 18 seconds (healthy)
compose-lab-db-1         postgres:16-alpine    "docker-entrypoint.s…"   db         1
1 minutes ago   Up 11 minutes (healthy)   5432/tcp
compose-lab-frontend-1   nginx:alpine          "/docker-entrypoint.…"   frontend   1
9 seconds ago   Up 7 seconds              0.0.0.0:8080->80/tcp, :::8080->80/tcp
sonya@fedora-4:~/compose-lab$
```

Создаются данные в базе командой “docker compose exec db psql -U user -d mydb -c "INSERT INTO items (name) VALUES ('apple'), ('banana'), ('cherry');"". Проверяется работа цепочки командой “curl localhost:8080/api/items”, выводятся данные из базы через nginx и backend.

Масштабируется backend командой “docker compose up -d --scale backend=3”. Запускаются три копии сервиса. Проверяется командой “docker compose ps” видны три контейнера backend.

```
sonya@fedora-4:~/compose-lab$ curl localhost:8080/api/items
[{"id":1,"name":"apple"},{"id":2,"name":"banana"},{"id":3,"name":"cherry"}]
sonya@fedora-4:~/compose-lab$ docker compose up -d --scale backend=3
[+] Running 5/5
 ✓ Container compose-lab-db-1        Healthy                                  0.0s
 ✓ Container compose-lab-backend-1   Heal...                                   0.0s
 ✓ Container compose-lab-backend-3   Heal...                                  11.5s
 ✓ Container compose-lab-backend-2   Heal...                                  11.5s
 ✓ Container compose-lab-frontend-1  Run...                                    0.0s
sonya@fedora-4:~/compose-lab$ docker compose ps
NAME                     IMAGE                 COMMAND                  SERVICE    C
REATED          STATUS                    PORTS
compose-lab-backend-1    compose-lab-backend   "python app.py"          backend    2
 minutes ago    Up 2 minutes (healthy)
compose-lab-backend-2    compose-lab-backend   "python app.py"          backend    1
6 seconds ago   Up 16 seconds (healthy)
compose-lab-backend-3    compose-lab-backend   "python app.py"          backend    1
6 seconds ago   Up 15 seconds (healthy)
compose-lab-db-1         postgres:16-alpine    "docker-entrypoint.s…"   db         1
3 minutes ago   Up 13 minutes (healthy)   5432/tcp
compose-lab-frontend-1   nginx:alpine          "/docker-entrypoint.…"   frontend   2
 minutes ago    Up 2 minutes              0.0.0.0:8080->80/tcp, :::8080->80/tcp
sonya@fedora-4:~/compose-lab$
```

Командой “docker volume ls” просматриваются все тома. Останавливается стек командой “docker compose down” контейнеры и сети удаляются. Останавливается стек с удалением данных командой “docker compose down -v”, том тоже удаляется. Очищается мусор командой “docker system prune -f”, удаляются неиспользуемые образы и кэш.

Сети изолируют трафик и дают DNS имена контейнерам. Volumes сохраняют данные при удалении контейнеров. Docker-compose управляет несколькими сервисами через один файл. Healthcheck проверяет готовность сервиса. Масштабирование запускает несколько копий.